

MeetAI - Technical Documentation Report

1. Project Overview

MeetAI is an AI-powered meeting and video conferencing SaaS platform.

Main Purpose & Problem Solved: The core purpose of MeetAI is to provide users with the ability to host video meetings that feature integrated, context-aware AI agents. Instead of simply recording a meeting, MeetAI actively connects an OpenAI Realtime Voice AI agent to the ongoing video call. This solves the problem of manual note-taking and lack of post-meeting intelligence by automatically capturing transcripts, generating AI summaries, and even allowing users to chat with an AI representation of the meeting afterward to ask follow-up questions.

Type of System: Full-stack SaaS Web Application / Dashboard.

2. System Functionality

Here are the major features and behaviors of the application:

- **Authentication & User Management:**
 - **What it does:** Allows users to sign up, log in, and manage their profiles.
 - **How it works:** Uses `better-auth` combined with a Drizzle ORM Postgres database adapter. It supports GitHub and Google OAuth, as well as traditional email/password credentials.
 - **Modules:** `src/lib/auth.ts` , `src/modules/auth` .
- **Custom AI Agent Creation:**
 - **What it does:** Users can define and create personal AI agents with specific instructions and behaviors.
 - **How it works:** Users define specific `instructions` for the AI which are saved in the `agents` table in the database.
 - **Modules:** `src/modules/agents` , `src/db/schema.ts` (`agents` table).
- **Video Meetings with AI Voice Agents:**
 - **What it does:** Users can initiate video calls where an AI agent joins as a participant to listen, transcribe, and speak in real-time.
 - **How it works:** Powered by Stream Video SDK. When a session starts (`call.session_started` webhook), the backend connects the OpenAI Realtime API to the active Stream Call. The agent utilizes the predefined instructions to guide its tone and interaction style using Voice Activity Detection (VAD).
 - **Modules:** `src/modules/call` , `src/modules/meetings` , `src/app/api/webhook/route.ts` .
- **Automated Meeting Processing (Summarization):**
 - **What it does:** Post-meeting, the platform processes the transcript to generate an intelligent summary.
 - **How it works:** Once a meeting ends and transcription is ready (`call.transcription_ready` webhook), an event is dispatched to **Inngest** (background job queue). Inngest fetches the transcript, matches speakers (identifying users vs. agents), and uses an OpenAI agent (`@inngest/agent-kit`) to write a narrative summary, which is saved back to the database.
 - **Modules:** `src/inngest/function.ts` , `src/app/api/inngest/route.ts` .
- **Post-Meeting AI Chat (Interactive Revisit):**

- **What it does:** Users can continue chatting with the AI agent inside the context of a completed meeting.
 - **How it works:** Uses Stream Chat SDK. Incoming chat messages trigger a `message.new` webhook. The backend pulls the meeting summary and the agent's original instructions, looks at the last 5 messages for context, and generates a response using `gpt-4o`, injecting the reply back into the Stream Chat channel seamlessly.
 - **Modules:** `src/app/api/webhook/route.ts` (Stream Chat webhook).
- **Monetization & Premium Upgrades:**
 - **What it does:** Provides a checkout flow for paid/premium tier access.
 - **How it works:** Integrated with Polar (`@polar-sh/better-auth`).
 - **Modules:** `src/modules/premium`, `src/lib/polar.ts`.

3. Architecture Explanation

Overall Architecture Workflow:

- **Frontend Structure:** Built on Next.js 16 (App Router) using React 19. It uses Radix UI and Tailwind CSS for the design system. The architecture is feature-sliced, meaning each major capability (dashboard, auth, meetings, call, agents) has its own directory inside `src/modules/`.
- **Backend Structure:** The primary backend utilizes Next.js App Router API Routes (`src/app/api/`) for webhooks and standard tRPC routers (`src/trpc/`) for strongly-typed client-to-server data fetching.
- **Database Structure:** A Serverless Postgres database hosted on Neon, accessed strictly via Drizzle ORM. The schema defines `user`, `session`, `account`, `agents`, and `meetings`.
- **API Flow (Data Fetching):** Uses tRPC (`@trpc/react-query`) with TanStack Query. The frontend queries the tRPC backend routers (`src/trpc/routers`), ensuring full end-to-end type safety.
- **Authentication Flow:** `better-auth` handles the session lifecycle. A user logs in via the UI, `better-auth` sets secure HTTP-only cookies, and standard requests automatically identify the user server-side.
- **Integrations & Event Flow:**
 - **Stream (Video/Chat):** The client directly talks to Stream for low-latency A/V and chat.
 - **Webhooks:** Stream fires webhooks to `src/app/api/webhook/route.ts` on events (call ended, transcription ready).
 - **Inngest:** The webhooks push async jobs to Inngest (`src/inngest/function.ts`) to handle heavy LLM processing without blocking external API responses.
 - **OpenAI:** Used both in real-time via Stream integration, inside Inngest for summarization, and directly via standard REST for the post-meeting chat.

4. Technologies Used

Technology	Purpose	Where It Is Used
Next.js 16 (App Router)	Full-stack framework	Core framework, UI routing, API routes (<code>src/app</code>)
React 19	Frontend UI Library	All UI components
Tailwind CSS 4	Styling	Global styles, component utility classes
Radix UI / CMDK	Accessible UI Primitives	Headless UI elements (<code>src/components/ui</code>)
Drizzle ORM	Database ORM	Schema definition and database queries (<code>src/db</code>)

Neon	Serverless Postgres Database	Primary datastore (via @neondatabase/serverless)
Better Auth	Authentication	User sessions and OAuth integration (src/lib/auth.ts)
tRPC	End-to-end typed APIs	Client/Server data fetching (src/trpc)
Stream Video & Chat	Communication SDK	Real-time video conferencing and chat messaging
OpenAI (Realtime / GPT-4o)	AI Models & Voice	Voice agents in calls, text summarization, and chat
Inngest	Background Jobs	Asynchronous transcript processing and summarization
Polar	Monetization	Subscription & payment processing (src/lib/polar.ts)

Why these technologies? The stack perfectly caters to a highly scalable, real-time AI SaaS. Next.js + tRPC provides high developer velocity. Stream avoids the immense complexity of building WebRTC from scratch. OpenAI Realtime handles the extreme latency requirements of conversational voice AI. Inngest prevents heavy AI summarization tasks from causing server timeouts.

5. File & Module Breakdown

- `src/app/` - The Next.js App Router entry points (pages, layouts).
 - `api/webhook/route.ts` - **Critical:** Handles all incoming events from Stream (Meeting start/end, chat messages).
 - `api/inngest/route.ts` - Entry point for Inngest background workers to poll and run tasks.
- `src/components/` - Shared, reusable frontend elements.
 - `ui/` - Highly customized Radix UI components based on shadcn/ui.
 - `data-table.tsx` / `data-pagination.tsx` - Reusable complex functional UI.
- `src/db/` - Database logic.
 - `schema.ts` - Drizzle tables defining the shape of users, agents, and meetings.
- `src/inngest/` - Background task definitions.
 - `function.ts` - Houses the `meetings/processing` function that uses AgentKit to summarize transcripts.
- `src/lib/` - Shared utility wrappers.
 - `auth.ts` - Better Auth configuration.
 - `stream-video.ts` / `stream-chat.ts` - Stream client initializations.
- `src/modules/` - Distinct feature areas (Feature-Sliced Design).
 - `agents/` - UI and server procedures for creating/editing AI agent personalities.
 - `meetings/` - UI and server procedures for listing and viewing meeting history/summaries.
 - `call/` - The active video interface components and state.
- `src/trpc/` - tRPC setup.
 - `routers/_app.ts` - The root router combining feature sub-routers.

6. Key Logic Explanation

- **Realtime AI Agent Integration (The Call Flow):**
 1. User creates a meeting linked to a specific customized AI Agent.
 2. User joins the Stream video call. Stream fires `call.session_started` to the webhook.
 3. The webhook identifies the agent from the DB.
 4. The server connects an OpenAI Realtime model directly to the Stream call using `streamVideo.video.connectOpenAi()` , passing the agent's custom instructions and configuring Voice Activity Detection (VAD).
 5. The AI listens and speaks organically inside the video call.
 6. User ends the call. Stream fires `call.session_ended` .
- **Asynchronous Summarization (The Processing Flow):**
 1. After the call, Stream completes audio-to-text processing and fires `call.transcription_ready` .
 2. The webhook triggers an Inngest background event (`meetings/processing`).
 3. The Inngest function starts in the background. It downloads the JSONL transcript file.
 4. It matches the `speaker_id` in the transcript to the respective database IDs for users and the AI agent to give the transcript proper names.
 5. It passes the parsed transcript to a specifically prompted `summarizer` agent powered by GPT-4o.
 6. The resultant markdown summary is saved to the `meetings` database row, marking it as "completed".

7. Data Flow

1. **Client $\rightarrow$ Server:** React frontend sends typed payloads via tRPC to `src/trpc/routers` .
2. **Server $\rightarrow$ Database:** tRPC procedures query/mutate the Neon Postgres database using Drizzle ORM.
3. **External Events $\rightarrow$ Server:** Stream SDK dispatches HTTP POST requests to `src/app/api/webhook` .
4. **Server $\rightarrow$ Background Queue:** The webhook sends an event payload to Inngest via `inngest.send()` .
5. **Queue $\rightarrow$ External API $\rightarrow$ Database:** Inngest fetches transcription Data from Stream, pushes it to OpenAI, gets the response, and writes the summary to the Neon database.
6. **Server $\rightarrow$ Client Response:** All successful real-time state changes rely on Stream SDK websockets or standard React Query invalidation loops via tRPC.

8. Current Limitations

- **Transcription Dependency:** Summarization relies heavily on the quality and format of Stream's default transcription JSONL format. If the speaker ID mapping fails, the summary might lack necessary context of "who said what."
- **Background Processing Latency:** Meeting summarization is entirely disconnected from the frontend lifecycle, meaning users have to wait an indeterminate amount of time or refresh the page to see the generated summary once a call finishes.
- **Clock Skew Patch:** There is an explicit monkey-patch in the webhook route (`iam: Math.floor(Date.now() / 1000) - 60`) for generating Call Tokens, indicating an ongoing issue with Stream/Server clock synchronization.

9. Future Enhancements

Enhancement Idea	Description	Status
------------------	-------------	--------

Multi-Agent Memory Sync (Personal AI Assistants)	Each user has an individual "personal AI" that retains memory of their schedules, prior meeting discussions, and plans. When multiple users join a meeting, their personal AIs sync relevant context, schedules, and prior shared knowledge to the main meeting AI.	Planned
Contextual Role Awareness	The meeting AI will explicitly understand the specific roles of each participant in the meeting (e.g., Project Manager, Lead Developer) and anticipate/expect specific outputs, updates, or action items based on those roles.	Planned
Real-time Fact-Checking & Knowledge Retrieval	The AI active in the meeting can instantly query company wikis, Jira, or past meeting transcripts during the call to verify claims or pull up requested data points in real-time, displaying them in a shared meeting sidebar.	Proposed
Sentiment & Engagement Analysis Dashboard	Post-meeting analytics that gauge the overall tone, engagement levels, and talk-time distribution of participants to help teams improve communication dynamics.	Proposed
Automated Action Item Delegation & Follow-up	AI automatically extracts action items, assigns them based on the detected user roles, and follows up with participants via Slack/Email/Stream Chat before the next scheduled sync.	Proposed
Cross-Language Real-time Translation	The AI agent facilitates global meetings by acting as a real-time translator, allowing participants to speak their native language while providing seamless audio or subtitle translation for others.	Proposed

10. Final Summary

MeetAI is a highly sophisticated, modern Next.js 16 SaaS application that effectively melds video conferencing (Stream) with real-time generative voice AI (OpenAI Realtime API). By employing a robust architecture featuring tRPC for typed client-server communication, Drizzle/Neon for serverless data persistence, and Inngest for resilient background task processing, the platform reliably acts as an automated virtual assistant that attends calls, converses naturally, and provides intelligent, asynchronous transcript summarizations.